agenbits

🧾 Project : `agenbits`

📌 Project Overview

agenbits is a Python package designed to simplify ingestion of remote data files for machine learning workflows. It fetches files from URLs (such as Azure Blob Storage or public links), converts them into binary `.bin` format temporarily, and then decodes them back into usable formats (like `pandas.DataFrame`, `PIL.Image`, `numpy arrays`, etc.) via a clean API.

✅ Key Features

- 🔗 **Fetch from URL**: Supports remote file downloads (e.g., CSV, XLSX, PNG, MP4, WAV).
- 💾 **Temporary Binary Storage**: Converts input to `.bin` files stored locally for model-ready input.
- 🔍 **Auto Format Detection**: Detects file format when not explicitly provided.
- 📂 **Modular API Access**:

  ```
  from agenbits import Excel, ImageFile, Audio, Text, Video, Binary
  ```

- 🧠 **Easy ML Integration**: Returns clean Python objects usable in ML pipelines.

Blackbox 📦

⚙️ How `agenbits` Works — Black Box Workflow Overview

The `agenbits` library transforms remote data into machine learning–ready formats through a modular and consistent pipeline. Below is a step-by-step breakdown of how data flows through the system:

🧭 Step-by-Step Workflow

✅ Step 1: User Calls Format Interface

You import and call a top-level interface like this:

```
from agenbits import ImageFile
img = ImageFile("https://example.com/sample.jpg")
```

This triggers format-specific logic inside `agenbits/interfaces/image.py`.

⬇️ Step 2: Downloader Fetches Data from URL

```
raw_data = fetch_data_from_url(url)
```

- Located in: `core/downloader.py`
- Uses the `requests` library to fetch file bytes from the provided URL.
- Throws a clear error if the request fails.

💾 Step 3: Binary File Saved to Temp Folder

```
binary_path = save_binary_file(raw_data)
```

- Located in: `core/converter.py`

- Creates a local `.bin` file in a temporary `tempdata/` directory.
- Ensures data is locally cached and ready for decoding by format-specific logic.

🕵️ Step 4: (Optional) File Type Detection

```
file_type = detect_file_type_from_url(url)
```

- Located in: `core/detector.py`
- Used when decoding depends on knowing the MIME type.
- For example, Excel files can be `.csv` or `.xlsx`, and are handled accordingly.

🔓 Step 5: Binary File Decoded Into Python Object

```
data = load_binary_file(binary_path)
```

- Located in: `core/decoder.py`
- Opens and reads the local `.bin` file.
- Format-specific modules like `image.py`, `excel.py`, `audio.py` then parse the bytes into native Python objects (e.g., `PIL.Image`, `pandas.DataFrame`, `numpy.ndarray`, etc.).

📤 Step 6: Return ML-Ready Python Object

- The final return is **not raw bytes**, but an **interpreted, structured** object:
  - `.ImageFile()` → returns a `PIL.Image.Image`
  - `.Excel()` → returns a `pandas.DataFrame`
  - `.Audio()` → returns a `numpy.ndarray`
  - `.Binary()` → returns file path or raw bytes
  - `.Video()` → returns path to the local `.mp4` file (for `cv2.VideoCapture()` or similar)

🔁 Summary: What Happens Behind the Scenes

Stage	Component	Role
Input	`interfaces/*.py`	Entry point – user calls `ImageFile(url)` etc.
Download	`core/downloader.py`	Fetches bytes from URL
Convert	`core/converter.py`	Stores the bytes as a temporary `.bin` file
Detect	`core/detector.py`	(If needed) detects file type from extension/MIME
Decode	`core/decoder.py` + interface logic	Converts binary to structured data (e.g., image/audio)
Output	Python object	ML-ready data for downstream pipelines

💡 Design Principles

- **Modular**: Separated into `core/`, `interfaces/`, and `utils/`
- **Scalable**: Easy to add more formats (PDF, ZIP, etc.) later
- **ML-Friendly**: Outputs data in formats that ML/DL models can use directly
- **PyPI-Ready**: Structured for public installation and continuous delivery

---

Would you like this in a PDF format for submission or wrapped into a README section as well?

📂 Folder Structure(no need to implement this in your machines)

```
agenbits/
│
├── agenbits/                # Main package
│   ├── __init__.py
│   ├── core/                # Internal logic
│   │   ├── __init__.py
│   │   ├── downloader.py      # fetch_data_from_url
│   │   ├── converter.py       # save_binary_file
│   │   ├── detector.py        # detect_file_type_from_url
│   │   └── decoder.py         # load_binary_file
│   │
│   ├── interfaces/          # User-facing APIs
│   │   ├── __init__.py
│   │   ├── excel.py
│   │   ├── image.py
│   │   ├── audio.py
│   │   ├── text.py
│   │   ├── video.py
│   │   └── binary.py
│   │
│   └── utils/
│       ├── __init__.py
│       └── file_utils.py      # extension utilities
│
├── tests/                   # Unit tests
├── examples/                # Usage examples
├── setup.py                 # PyPI configuration
├── README.md                # Documentation
├── LICENSE                  # MIT License
└── .gitignore
```

🔧 Installation

```
pip install agenbits
```

📚 Usage Examples

1. Load an Excel/CSV file

```
from agenbits import Excel
df = Excel("https://example.com/sample.csv")  # Auto detects CSV
print(df.head())
```

2. Load an Image

```
from agenbits import ImageFile
img = ImageFile("https://example.com/image.png")
img.show()
```

3. Load Audio

```
from agenbits import Audio
audio_array = Audio("https://example.com/audio.wav")
```

4. Load Video

```
from agenbits import Video
path = Video("https://example.com/video.mp4")  # Returns binary video path
```

📦 Supported File Types

Type	Extensions
Excel	`.csv`, `.xlsx`, `.tsv`
Image	`.jpg`, `.jpeg`, `.png`
Audio	`.wav`, `.flac`
Video	`.mp4`
Text	`.json`, `.txt`, `.xml`, `.sgm`
Binary	`.bin`, `.pt`, `.tfrecord`
Graph	`.edgelist`

🔄 Versioning

- First public version: `0.1.0`
- Patch submitted as `0.1.1` to fix `ModuleNotFoundError` for `core` imports.

👨‍💻 Authors & Maintainers

- **Enos Uppada**
- Email: `enosuppada2005@gmail.com`
- GitHub: github.com/dunkdumb

Let me know if you want a `.docx` or `.pdf` version of this.